AICTE - EDUSKILLS VIRTUAL INTERNSHIP

PROJECT REPORT

"File Metadata Analyzer"

SUBMITTED BY:

Student Name: Pamera Patel
Roll Number: N/A
Program: B.Tech
Branch: Computer Science

SUBMITTED TO:

EduSkills Academy

Internship Details:

Internship Duration: 8 Weeks (Oct 2026 - Oct 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

IP: 43.241.64.136
Generated: 12-Oct-2026 11:13:51 AM IST
Device: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/...

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Pamera Patel** , Roll No. **N/A**, a student of **EduSkills Academy**, has successfully completed **8-Week (Oct 2026 - Oct 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled **"File Metadata Analyzer"**. His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera
Mentor, EduSkills Academy
Date: 12-10-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Pamera Patel
(Signature)
Date: 12-10-2026

IP: 43.241.64.136
Generated: 12-Oct-2026 11:13:51 AM IST
Device: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/...

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagarabala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **Computer Science** for their encouragement and support.

ABSTRACT

This project implements a file analysis engine to generate structural metadata from text input. The engine processes text line-by-line, employing various analytical components including line enumeration, blank line classification, word frequency counting, character analysis, and longest line tracking. The solution adheres to strict technical constraints, utilizing Java's List and Scanner APIs for manual string building to ensure high-performance JSON generation without external dependencies.

IP: 43.241.64.136
Generated: 12-Oct-2026 11:13:51 AM IST
Device: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/...

Table of Contents

1. Introduction 2
2. Organization Profile 3
3. Internship Overview 4
4. Problem Statement 5
5. Methodology 6
6. Tools & Technologies 7
7. Implementation 8
8. Learning Outcomes 11
9. Challenges & Solutions 12
10. Conclusion 13
11. Future Scope 14
12. References 15
13. Annexure 16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Java Full Stack With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

• Internship Mode: **Online / Virtual**

• Duration: **8 Weeks**

• Platform Used: **EduSkills LMS**

• Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The File Metadata Analyzer is a robust engine designed to process text files and extract valuable structural metadata. Operating on a line-by-line basis, it provides comprehensive insights into file content through several analytical modules. These modules encompass the enumeration of lines, classification of blank lines, calculation of word frequencies per line, detailed character counts for each line, and the tracking of the longest line encountered. The project emphasizes high-performance JSON output generated through manual string construction using Java's List and Scanner APIs, strictly avoiding external libraries to meet technical constraints. This approach ensures efficient and dependency-free metadata generation.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the File Metadata Analyzer was approached systematically. Initially, the core requirement of processing text input line-by-line was established. Subsequently, individual analytical components were designed and implemented as distinct modules. This included a Line Enumerator for basic indexing, a Blank Line Classifier to differentiate content from empty lines, a Word Frequency Counter for word analysis per line, a Character Analysis component for detailed character counts, and a Longest Line Tracker to monitor line length dynamics. Each component was developed to operate independently and contribute to the overall metadata generation, with a focus on efficient manual string manipulation using Java's List and Scanner APIs to produce JSON output.

6. Tools & Technologies Used

Programming Language: Java
Libraries: Java Collections Framework (List), Java I/O (Scanner)
Concepts: String Manipulation, File Processing, JSON Generation (manual string building)

7. Implementation & Results

Problem 1: Line Enumerator

Description: Problem Statement: Develop a component that processes a stream of text, outputting a sequential numerical index for every line encountered. This serves as a foundational mechanism for organizing and referencing data within the analysis.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        int count = 0;

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            // TODO: Increment count and add a JSON string {"line":["X"]} to results
            count++;
            results.add("{\"line\":[\"" + count + "\"]}");
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 2: Blank Line Classifier

Description: Problem Statement: Implement functionality to distinguish between lines containing actual content and those that are blank or consist solely of whitespace. This classification is crucial for filtering and understanding the structural composition of the text.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            // TODO: Determine if line is blank. Add {"type":["blank"]} or {"type":["conte
            if (line.trim().isEmpty()) {
                results.add("{\"type\":[\"blank\"]}");
            } else {
                results.add("{\"type\":[\"content\"]}");
            }
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 3: Word Frequency Counter

Description: Problem Statement: Design a mechanism to accurately count the number of words present on each individual line of text. Words are to be defined as sequences of non-whitespace characters separated by one or more spaces.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();
            // TODO: Calculate word count and add {"words":["X"]} to results
            int count = 0;

            if (!line.isEmpty()) {
                count = line.split("\\s+").length;
            }

            results.add("{\"words\":[\"" + count + "\"]}");
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 4: Character Analysis

Description: Problem Statement: Create a component responsible for calculating the total number of characters within each processed line. This count should include all characters, irrespective of their type, including whitespace and special symbols.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            // TODO: Get length and add {"chars":["X"]} to results
            int count = line.length();

            results.add("{\"chars\":[\"" + count + "\"]}");

        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 5: Longest Line Tracker

Description: Problem Statement: Develop a feature that continuously monitors and reports the length of the longest line encountered up to the current point in the file processing. For each line processed, the output should reflect the maximum line length observed since the initiation of the analysis.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        int maxSoFar = 0;

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            // TODO: Update maxSoFar and add {"max":["X"]} to results
            int length = line.length();

            if (length > maxSoFar) {
                maxSoFar = length;
            }

            results.add("{\"max\":[\"" + maxSoFar + "\"]}");
        }

        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

8. Learning Outcomes

• Gained practical experience in line-by-line text file processing using Java.
• Developed proficiency in manual string manipulation for structured data generation (JSON).
• Understood the implementation of various text analysis algorithms (counting, classification, tracking).
• Reinforced the importance of adhering to technical constraints for performance and dependency management.

9. Challenges Faced & Solutions

Challenge: Implementing efficient JSON generation using only manual string building without relying on libraries like Jackson or Gson. Solution: Meticulously constructed JSON strings by concatenating appropriate characters (like '{', '}', ':', ',', '"') and using Java's List to manage and order the generated data elements before final string assembly.
Challenge: Handling edge cases in word definition and blank line detection (e.g., lines with only tabs or multiple spaces). Solution: Implemented robust parsing logic within the Scanner and String manipulation methods to accurately identify word boundaries based on one or more spaces and define blank lines based on the absence of non-whitespace characters.

10. Conclusion

The File Metadata Analyzer successfully addresses the requirement of generating detailed structural metadata from text files through a modular, line-by-line processing engine. Adherence to manual string building with Java's core APIs ensured efficient, dependency-free JSON output, demonstrating a practical approach to performance-critical data processing.

11. Future Scope

• Extend the analysis to include more sophisticated content classification (e.g., sentiment analysis, topic modeling).
• Implement support for additional file formats beyond plain text.
• Enhance the output to include statistical summaries (e.g., average line length, distribution of line types).

12. References

Java SE Documentation. Oracle Corporation. (Year of access).
Effective Java. Joshua Bloch. Addison-Wesley Professional. (Relevant Edition).
Data Structures and Algorithms in Java. Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser. Wiley. (Relevant Edition).

13. Annexure

- Internship Completion Certificate
- Offer Letter